

L01 · CORALNPU 教学仓

L01 · RV32I 单周期核

作业说明 —— 任务拆解、验收标准与调试方法

对应上游仓库: google/coralnpu (RTL、文档、测试程序都来自这里)

学习方式: 读规格 → 写 RTL → ./learn check 对拍 → 修复

文档版本: 2026-09-12 · 生成命令: ./learn pdf

目录

- [L01 作业说明](#)
 - [0. 交付物](#)
 - [1. 接口契约 \(不能改\)](#)
 - [2. 任务分解 \(建议顺序\)](#)
 - [3. 调试方法 \(按顺序用\)](#)
 - [4. 常见坑清单](#)
 - [5. 验收标准](#)
 - [6. 进阶挑战 \(可选, 不计入验收\)](#)
 - [7. 学习记录 \(建议提交\)](#)
 - [9. 复用 L00 的成果: 在 cpu_core 里实例化 alu32](#)

L01 作业说明

0. 交付物

如果「译码、立即数、位宽、时钟边沿、寄存器堆」这些词让你没底, 先跑 ./learn diag 定位缺哪一层, 再上预备课 P0 (模块 C/D/E 正好覆盖这些内容): ./learn start P0。

交付物	位置	说明
你的核心实现	course/lessons/ L01_scalar/rtl/ cpu_core.sv	只有这一个文件需要你写
自检输出	终端里 ./learn check L01 的最后一行	必须是 5/5 个用例通过
学习记录	course/work/L01/记 录.md (自己新建)	见第 7 节

不需要改动测试平台、黄金模型和链接脚本——它们扮演「客户与验收方」的角色。

1. 接口契约（不能改）

```
module cpu_core (  
    input logic          clk, rst,  
    output logic [31:0] io_imem_addr,    // 取指地址  
    input logic [31:0] io_imem_rdata,    // 当拍返回的指令  
    output logic [31:0] io_dmem_addr,    // 访存地址  
    output logic [3:0] io_dmem_wmask,    // 字节写掩码  
    output logic [31:0] io_dmem_wdata,    // 写数据（已按地址对齐）  
    output logic          io_dmem_we,     // 写使能  
    input logic [31:0] io_dmem_rdata,    // 当拍返回的整字数据  
    output logic          o_retire_valid, // 本周期有指令退休  
    output logic [31:0] o_retire_pc, o_retire_inst,  
    output logic [4:0] o_retire_rd,  
    output logic [31:0] o_retire_wdata,  
    output logic          o_halted  
);
```

约定：

1. 存储器端口是**组合读**：地址当拍给出，数据当拍返回；
2. `io_dmem_we` 有效时，测试平台在该时钟上升沿按 `io_dmem_wmask` 写入 DTCM；
3. 复位时 `pc = 0`，32 个通用寄存器全部清零；
4. 每条退休指令都要在 `o_retire_*` 上出现一次；
5. 遇到 `mpause (0x08000073)`、`ebreak`、`ecall` 时置起 `o_halted`，之后不再产生 trace。

`o_retire_rd = 0` 表示「这条指令不写寄存器」，此时 `o_retire_wdata` 必须是 0。

2. 任务分解（建议顺序）

每一步都对应一次 `./learn check L01`，失败位置会从第 1 条指令往后退。

Step 1：让 PC 会走，让 trace 有内容（约 20 分钟）

- 立即数：先把 `imm_i` 和 `imm_u` 写对（`01_arith.S` 的第一条是 `auipc`）；
- 控制：`reg_we`、`alu_result`；
- trace：`o_retire_rd = reg_we ? rd : 0`, `o_retire_wdata = reg_we ? wb_data : 0`。

自检: `./learn check L01 --only 01`, 第一条不一致应该从第 1 条变成第 6 条左右。

Step 2 : 算术与逻辑全通 (约 40 分钟)

补齐 OP-IMM 与 OP 两类: 加减、逻辑、移位、比较。注意 `slt/sltu` 和 `slti/sltiu` 的有符号/无符号区别, 以及移位量只取低 5 位。

自检: `./learn check L01 --only 01` 显示通过。

Step 3 : 分支与循环 (约 40 分钟)

补齐 6 种分支 + `jal` + `jalr`。B 型立即数的位序按图抄, 不要凭记忆。

自检: `./learn check L01 --only 02` 通过 (这是一个 1..10 求和的循环)。

Step 4 : 访存 (约 40 分钟)

- 存储: `sb/sh/sw` 的写掩码与数据对齐;
- 载入: 整字右移 `8*addr[1:0]` 后取低 1/2 字节, 再做符号/零扩展。

自检: `./learn check L01 --only 03` 通过。

Step 5 : 函数调用 (约 20 分钟)

如果 Step 3 的 `jal/jalr` 写对, 这一步通常不需要新代码——它主要验证栈上的 `ra` 保存与恢复在你的核上工作正常。

自检: `./learn check L01 --only 04` 通过。

Step 6 : 跑真实 C 程序 (约 30 分钟)

`05_c_fib.c` 是用真实交叉编译器编译的, 会用到 `-O0` 生成的栈帧访问、`lui/addi` 取地址、循环分支。如果失败, 先看第一个不一致的指令。

自检: `./learn check L01 --only 05` 通过。

Step 7 : 全量验收

`./learn check L01`

期望输出:

[通过] 01_arith.S	58 条指令, DTCM 全量一致, 58 周期
[通过] 02_branch.S	70 条指令, DTCM 全量一致, 70 周期
[通过] 03_mem.S	42 条指令, DTCM 全量一致, 42 周期
[通过] 04_call.S	219 条指令, DTCM 全量一致, 219 周期
[通过] 05_c_fib.c	254 条指令, DTCM 全量一致, 254 周期

5/5 个用例通过

3. 调试方法（按顺序用）

第一招：读检查器的错误。 它会告诉你第几条指令、期望值、实际值、该查哪个知识点。

第二招：只跑一个用例。

```
./learn check L01 --only 01
```

第三招：自己看黄金模型的 **trace。** 想知道「正确的时候长什么样」：

```
python3 - <<'PY'
import sys; sys.path.insert(0, 'course')
from golden.rv32i import Core
from tools import build_program
b = build_program.build('course/lessons/L01_scalar/tests/programs/01_arith
c = Core(); c.load_elf(b.elf)
for r in c.run()[:10]:
    print(f"{r.pc:08x} {r.inst:08x} rd={r.rd:02d} wdata={r.wdata:08x}")
PY
```

第四招：加打印。 在 RTL 里临时加一行，只看可疑指令：

```
always_ff @(posedge clk) if (!rst) $display("pc=%h inst=%h alu=%h rd=%0d we
pc, inst, alu_y, rd, reg_we);
```

第五招：看波形。 需要装波形工具（`sudo apt-get install gtkwave`），让测试平台加 `+WAVES=1` 生成 VCD，再逐周期看时序。

卡住 30 分钟以上，就按顺序看提示：`./learn hint L01 1`、`./learn hint L01 2`……跑通 3 个用例之后，可以 `./learn unlock L01` 对照参考实现。

4. 常见坑清单

这份清单来自课程作者实际踩过的坑，逐条对应检查器可能给你的报错：

现象	原因	修法
一条 trace 都没有, 20 万周期超时	PC 没前进或 mpause 没停机	确认 <code>next_pc = pc+4</code> 且 <code>o_halted</code> 能置起
第 1 条 <code>auipc</code> 的 <code>wdata</code> 是 0	U 型立即数没左移 12 位/没加 PC	<code>imm_u = {inst[31:12], 12'b0}</code> , <code>pc + imm_u</code>
<code>sra</code> 结果高位补 0	用了 <code>\$signed(a) >>> b</code>	用符号填充掩码写法
<code>lb/lh</code> 读到错的字节	忘了按 <code>addr[1:0]</code> 右移整字	先 <code>word = rdata >> (8*addr[1:0])</code>
<code>sb</code> 覆盖了整字	写掩码恒为 <code>4'b1111</code>	掩码按 <code>addr[1:0]</code> 平移
<code>sltu</code> 结果不对	用了 <code>\$signed</code> 比较	无符号直接比较
分支总是跳/总是不跳	立即数拼装或比较方向错	对照图 5 抄 B 型位序
<code>jalr</code> 跳转地址错一位	忘了最低位清零	<code>(rs1 + imm) & ~32'd1</code>
死循环 (函数不返回)	<code>ra</code> 被覆盖没保存	调用前 <code>sw ra, 0(sp)</code>
仿真出现 X 值	组合逻辑有分支没赋值, 或复位不完整	给默认值、复位清零全部寄存器
iverilog 报 <code>sorry: ...</code> 警告	常量位选在 <code>always</code> 块里的优化提示	不影响功能; 也可以用拼接替代

5. 验收标准

1. `./learn check L01` 输出 5/5 个用例通过;
2. 全程没有修改测试平台 (`tests/`)、黄金模型 (`golden/`)、链接脚本 (`tests/link/`);
3. 能口头解释: 为什么 `jalr` 要清最低位、`lb` 为什么要先移整字、复位为什么要清零寄存器。

这三条里, 第 3 条最重要——它决定你是「跑通了」还是「学到了」。

6. 进阶挑战 (可选, 不计入验收)

- **挑战 1:** 给你的核加上 `mul/div` (M 扩展), 让 `-march=rv32im` 编译的程序也能跑 (这就提前做了 L03 的一部分);
- **挑战 2:** 支持非对齐的 `lw/sw` (提示: 拆成两次对齐访问或做字节重排, L02 会给规格);
- **挑战 3:** 把单周期改成两段流水 (取指 / 执行分离), 观察 `addi` 后面跟分支时需要停几拍;
- **挑战 4:** 用 `iverilog -g2012 -Wall` 打开全部警告, 把警告清零。

7. 学习记录（建议提交）

在 `course/work/L01/记录.md` 里写下：

1. 每个 Step 用掉的时间；
2. 你遇到的**第一个真正卡住你的 bug**：现象、你的猜测、最后是怎么定位的；
3. 对照上游 `Decode.scala`，说出你的译码表少支持了哪些东西。

第 2 条是这份作业最有价值的部分——它记录了你的调试方法有没有进步。

9. 复用 L00 的成果：在 `cpu_core` 里实例化 `alu32`

可以复用，而且检查器已经支持。L01 编译时会自动带上：

- 你正在写的 `course/lessons/L01_scalar/rtl/cpu_core.sv`；
- 同一目录下的其它 `.sv`（想把模块拆成多个文件时用）；
- 你在 L00 写的 `course/lessons/L00_setup/rtl/alu32.sv`。

所以只要在 `cpu_core.sv` 里实例化 `alu32` 就能直接用，不需要复制文件。代价也很直接：**L00 的 ALU 有 bug，L01 就会跟着失败**——这正是复用别人的（或自己过去的）模块时的纪律。

9.1 实例化语法

```
logic [31:0] alu_a, alu_b, alu_y;
logic [3:0]  alu_op;
logic       alu_zero;

alu32 u_alu (
    .a    (alu_a),
    .b    (alu_b),
    .op   (alu_op),
    .y    (alu_y),
    .zero(alu_zero)    // 单周期核用不到零标志，写成 .zero() 空接也可以
);
```

命名端口连接的写法 `.端口名(信号名)` 比按位置连接更安全：端口顺序变了也不会接错。

9.2 接口适配：指令字段 → ALU 操作码

`cpu_core` 手里是 `opcode/funct3/funct7`，`alu32` 要的是 0-9 的操作码，中间需要一个**适配器**（就是一段译码逻辑）：

指令	alu_op	说明
addi / add / 访存地址 / auipc	0 ADD	加法最常用，做默认值
sub	1 SUB	
slli / sll	2 SLL	移位量由 alu32 自己取 b[4:0]，所以直接把 imm_i 或 rs2 接进 b 即可
slti / slt	3 SLT	有符号比较（注意 \$signed，L00 已经处理好了）
sltiu / sltu	4 SLTU	无符号比较
xori / xor	5 XOR	
srlr / srl	6 SRL	
srai / sra	7 SRA	算术右移，L00 的符号填充写法
ori / or	8 OR	
andi / and	9 AND	

关键的两处：

- **inst[30]** 决定 add/sub 与 srl/sra: $\text{alu_op} = \text{inst}[30] ? \text{ALU_SUB} : \text{ALU_ADD};$
- **OP-IMM** 与 **OP** 的 **b** 操作数不同：前者接 imm_i，后者接 rs2_val。

9.3 资源复用：一个 ALU 干两件事

访存指令要算地址 $\text{rs1} + \text{imm}$ ，算术指令要算结果——这两件事都发生在同一个周期、而且不会同时需要（一条指令要么访存要么运算）。所以可以让**同一个 ALU** 两者都干：

```
assign io_dmem_addr = alu_y;    // 载入/存储的地址就是 ALU 的加法结果
```

但有三处加法**不该**塞进这个 ALU，否则要靠更多多路选择器去挤：

1. $\text{pc} + 4$ （自增，综合出来就是个小的递增器）；
2. $\text{pc} + \text{imm_b} / \text{pc} + \text{imm_j}$ （分支与跳转目标）；
3. 复用带来的输入多路选择会变长关键路径——**共享硬件不是免费的**。

这就是数据通路里最经典的权衡：**复用省面积，但要在输入端加多路选择器、可能拖慢频率**。哪边划算要看规模：共享一个大乘法器通常值，共享一个加法器往往不值。

9.4 怎么验证

```
./learn check L01                                # 检查 rtl/cpu_core.sv
./learn check L01 --from refs/cpu_core_reuse.sv  # 看课程自带的「复用版」参考实现
```

--from 可以指向任何文件，方便你把实现拆成几个版本对比着调。复用版参考实现与 refs/cpu_core.sv 功能等价，执行同样的 5 个用例并全部通过。

面向前一课的设计习惯：如果你打算复用 L00 的 ALU，最好在 L00 阶段就把接口定清楚（操作码含义、位宽、是否需要 zero 标志）。硬件里「先定接口、再写实现」比事后改接口便宜得多——这也是模块化设计的核心。